LAPORAN PRATIKUM PEKAN 4

STRUKTUR DATA

disusun Oleh:

ALIF MUHAMMAD IHSAN

NIM 2511531004

Dosen Pengampu : Dr.Wahyudi S.T, M.T

Asisten : M Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum **Struktur Data** ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar struktur data, implementasi algoritma, serta penerapan logika pemrograman dalam kehidupan nyata.

Dalam penyusunan laporan ini, penulis menyadari masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Ucapan terima kasih penulis sampaikan kepada:

- Dosen pengampu mata kuliah **Struktur Data** yang telah memberikan arahan dan bimbingan.
- Asisten praktikum yang senantiasa membantu dalam memahami materi dan menyelesaikan permasalahan teknis.
- Rekan-rekan mahasiswa yang turut berdiskusi dan bekerja sama selama kegiatan praktikum berlangsung.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari lebih lanjut mengenai struktur data.

Daftar Isi

KATA PENGANTAR.....i

BAB I.....3

PENDAHULUAN.....3

1.1 Latar Belakang.....3

1.2 Tujuan.....3

1.3 Manfaat Praktikum.....4

BAB II.....5

PEMBAHASAN.....5

2.1 Kode Program.....**Error! Bookmark not defined.**

2.2 Langkah Kerja.....**Error! Bookmark not defined.**

2.3 Analisis Hasil.....**Error! Bookmark not defined.**

BAB III.....**Error! Bookmark not defined.**

PENUTUP.....**Error! Bookmark not defined.**

3.1 Kesimpulan.....**Error! Bookmark not defined.**

3.2 Saran.....**Error! Bookmark not defined.**

DAFTAR PUSTAKA.....**Error! Bookmark not defined.**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengorganisasian data. Dengan adanya struktur data, proses penyimpanan, pengambilan, serta manipulasi data dapat dilakukan secara lebih efisien dan terstruktur. Pemahaman mengenai struktur data menjadi dasar bagi pengembangan algoritma yang optimal, sehingga sangat berpengaruh terhadap performa suatu program maupun sistem.

Melalui praktikum struktur data, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga mengimplementasikan langsung berbagai bentuk struktur data seperti array, linked list, stack, queue, dan tree dalam bahasa pemrograman. Praktikum ini bertujuan untuk melatih keterampilan analisis logika, pemecahan masalah, serta kemampuan debugging yang diperlukan dalam pengembangan perangkat lunak.

Selain itu, praktikum ini juga memberikan pengalaman nyata dalam menghubungkan teori dengan praktik, sehingga mahasiswa dapat memahami bagaimana suatu algoritma bekerja di balik layar dan bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi program. Dengan demikian, kegiatan praktikum struktur data menjadi sarana penting untuk membangun fondasi keilmuan yang kuat dalam bidang informatika.

1.2 Tujuan

1. Memahami konsep dasar struktur data dan penerapannya dalam pemrograman.
2. Melatih keterampilan dalam mengimplementasikan berbagai jenis struktur data seperti array, linked list, stack, queue, dan tree menggunakan bahasa pemrograman.
3. Mengembangkan kemampuan analisis logika dan pemecahan masalah melalui studi kasus yang diberikan dalam praktikum.
4. Melatih keterampilan debugging serta troubleshooting program agar dapat menemukan dan memperbaiki kesalahan secara sistematis.

1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data sehingga mahasiswa lebih mudah dalam memahami konsep yang abstrak
2. Meningkatkan keterampilan pemograman, khususnya dalam efesiensi algoritma dan pengelolaan data.
3. Menjadi bekal penting untuk mata kuliah lanjutan maupun proyek pengembangan perangkat lunak di masa depan.
4. Membantu mahasiswa memahami pentingnya pemilihan struktur data yang tepat dalam meningkatkan peforma program.

BAB II

PEMBAHASAN

2.1 Kode Progaram

a. Kode QueueArray_2511531004

```
package pekan4_2511531004;

public class queueArray_2511531004 {
        int front_1004,rear_1004,size_1004;
        int capacity_1004;
        int array_1004[];

        public queueArray_2511531004(int capacity) {
                this.capacity_1004 = capacity;
                front_1004 = this.size_1004 = 0;
                rear_1004 = capacity - 1;
                array_1004 = new int [this.capacity_1004];

        }

        boolean isFull_2511531004 (queueArray_2511531004 queue) {
                return (queue.size_1004 == queue.capacity_1004) ;

        }

        boolean isEmpty_2511531004 (queueArray_2511531004 queue) {
                return (queue.size_1004 == 0) ;

        }

        void enqueue_2511531004 ( int item ) {
                if (isFull_2511531004 (this))
```

```
                return ;
            this.rear_1004 = (this.rear_1004 + 1 ) % this.capacity_1004;
            this.array_1004 [this.rear_1004] = item;
            this.size_1004 = this.size_1004 + 1;
            System.out.println(item + " enqueued to queue");
        }
        int dequeue_1004 () {
            if (isEmpty_2511531004 (this))
                return Integer.MIN_VALUE;
            int item = this.array_1004[this.front_1004];
            this.front_1004 = (this.front_1004 + 1 ) % 
this.capacity_1004;
            this.size_1004 = this.size_1004 - 1;
            return item;
        }
        int front_1004() {
            if(isEmpty_2511531004(this) )
            return Integer.MIN_VALUE;

            return this.array_1004[this.front_1004];

        }
        int rear_1004() {
            if (isEmpty_2511531004(this))
                return Integer.MIN_VALUE;
            return this.array_1004 [this.rear_1004];

        }
        // mencetak elemen antrian
        void display_2511531004 () {
            int i_1004;
            if ( front_1004 == rear_1004) {
```

```
                System.out.println("\nAntrian kosong\n");
                return;
            }
            // kunjungi dari belakang dan cetak
            for (i_1004 = front_1004; i_1004 < rear_1004; i_1004++) {
                System.out.printf(" %d <-- " , array_1004[i_1004]);
            }
            return;
        }

}
```

b. Kode QueueArrayDriver_2511531004

```
package pekan4_2511531004;

public class queueArrayDriver_2511531004 {

    public static void main(String[] args) {
        queueArray_2511531004 queue = new queueArray_2511531004 (1000);
        queue.enqueue_2511531004(10);
        queue.enqueue_2511531004(20);
        queue.enqueue_2511531004(30);
        queue.enqueue_2511531004(40);
        System.out.println("item di depan " + queue.front_1004() );
        System.out.println("item di belakang " + queue.rear_1004() );
        System.out.println("tampilkan queue");
        queue.display_2511531004();
        System.out.println();
        System.out.println(queue.dequeue_1004() + " dihapus dari queue");
```

```
                System.out.println("item di depan " + queue.front_1004
() );
                System.out.println("item di belakang " +
queue.rear_1004() );
                System.out.println("tampilkan queue setelah satu data
dihapus");
                queue.display_2511531004();



        }

}
```

c. Kode QueueLinkedList_2511531004

```
package pekan4_2511531004;

import java.util.Queue;

import java.util.LinkedList;


public class queueLinkedList_2511531004 {

    public static void main(String[] args) {


        Queue<Integer> q_1004 = new LinkedList<> ();

        // menambah elemen ke dalam antrian


        for(int i = 0 ; i < 6 ; i ++)

            q_1004.add(i);
```

```
            // menampilkan ke dalam antrian
            System.out.println("elemen antrian " + q_1004);
            //untuk menghapus kepala antrian
            int hapus_1004 = q_1004.remove();
            System.out.println("elemen yang dihpaus " + hapus_1004);
            System.out.println(q_1004);
            //untuk melihat antrian di depan
            int depan = q_1004.peek();
            System.out.println("elemen terdepan " + depan);

            int banyak = q_1004.size();
            System.out.println("size antrian " + banyak);

    }

}
```

d. Kode iterasiQueue_2511531004

```
package pekan4_2511531004;
import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;
public class iterasiQueue_2511531004 {
    public static void main(String[] args) {
```

```
        Queue<String> q_1004 = new LinkedList<> () ;

        q_1004.add("Praktikum");
        q_1004.add("Struktur");
        q_1004.add("Data");
        q_1004.add("Dan");
        q_1004.add("Algoritma");
        Iterator<String> iterator = q_1004.iterator() ;
        while ( iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }
    }

}
```

e. Kode ReverseData_2511531004

```
package pekan4_2511531004;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class reverseData_1004 {
    public static void main(String[] args) {
```

```
        Queue<Integer> q_1004 = new LinkedList<Integer> ();
        q_1004.add(1);
        q_1004.add(2);
        q_1004.add(3);
        System.out.println("Sebelum reverse " + q_1004 );
        Stack<Integer> s_1004 = new Stack<Integer> () ;
        while (!q_1004.isEmpty()) {
            s_1004.push(q_1004.remove() );
        }
        while (!s_1004.isEmpty()) {
            q_1004.add(s_1004.pop());
        }



        System.out.println("data sesudah di reverse " + q_1004);
    }

}
```

2.2 Langkah Kerja

a) Langkah queueArray_2511531004

1. Inisialisasi

- Saat objek dibuat dengan new queueArray_2511531004(capacity):
    - capacity_1004 → kapasitas maksimal queue.
    - front_1004 = 0 → posisi awal elemen depan.
    - rear_1004 = capacity - 1 → posisi belakang di-set ke indeks terakhir.
    - size_1004 = 0 → jumlah elemen awal = 0.
    - array_1004[] → array kosong dengan ukuran sesuai kapasitas.

2. Mengecek Kondisi Queue

- isFull_2511531004 → true jika size == capacity.
- isEmpty_2511531004 → true jika size == 0.

3. Operasi Enqueue (menambah elemen)

java

void enqueue_2511531004(int item)

- Cek apakah queue penuh → jika penuh, berhenti.
- Update rear dengan (rear + 1) % capacity → supaya melingkar.
- Simpan item ke array[rear].
- Tambah size → size = size + 1.
- Cetak pesan bahwa item berhasil masuk.

4. Operasi Dequeue (menghapus elemen depan)

java

int dequeue_1004()

- Cek apakah queue kosong → jika kosong, return Integer.MIN_VALUE.
- Ambil elemen di array[front].
- Update front dengan (front + 1) % capacity.
- Kurangi size → size = size - 1.
- Return elemen yang dihapus.

5. Akses Elemen Depan & Belakang

- front_1004() → mengembalikan elemen di posisi depan.
- rear_1004() → mengembalikan elemen di posisi belakang.

6. Menampilkan Isi Queue

java

void display_2511531004()

- Jika front == rear, dianggap kosong.
- Jika tidak, loop dari front sampai rear dan cetak elemen dengan tanda panah <--.

b) queueArrayDriver_2511531004

1. Inisialisasi Queue

Membuat objek queue dengan kapasitas 1000.

Pada tahap ini, front = 0, rear = 999, dan size = 0.

2. Operasi Enqueue

Menambahkan elemen ke dalam queue:

enqueue(10) → elemen 10 masuk.

enqueue(20) → elemen 20 masuk.

enqueue(30) → elemen 30 masuk.

enqueue(40) → elemen 40 masuk.

Setelah empat kali enqueue, queue berisi [10, 20, 30, 40].

3. Menampilkan Elemen Depan dan Belakang

front() → menampilkan elemen paling depan (10).

rear() → menampilkan elemen paling belakang (40).

4. Menampilkan Isi Queue

- Memanggil display() untuk mencetak isi queue dari depan ke belakang.
- Output: 10 <-- 20 <-- 30 <--

5. **Operasi Dequeue**

- dequeue() → menghapus elemen paling depan (10).
- Queue sekarang berisi [20, 30, 40].

6. Menampilkan Kondisi Setelah Dequeue

- front() → elemen depan sekarang adalah 20.
- rear() → elemen belakang tetap 40.

- display() → menampilkan isi queue setelah penghapusan.

c) queueLinkedList_2511531004

1. **Inisialisasi Queue**

- Membuat objek Queue<Integer> q_1004 = new LinkedList<>();.
- Queue masih kosong.

2. **Menambah Elemen ke Antrian (Enqueue)**

- Menggunakan loop for(int i = 0; i < 6; i++) q_1004.add(i);.
- Elemen yang masuk: 0, 1, 2, 3, 4, 5.
- Queue sekarang berisi [0, 1, 2, 3, 4, 5].

3. **Menampilkan Isi Antrian**

- System.out.println("elemen antrian " + q_1004);.
- Output: elemen antrian [0, 1, 2, 3, 4, 5].

4. **Menghapus Elemen Depan (Dequeue)**

- Int hapus_1004 = q_1004.remove();.
- Elemen yang dihapus adalah 0 (karena queue bekerja dengan prinsip FIFO).
- Queue sekarang berisi [1, 2, 3, 4, 5].

d). iteratorQueue_2511531004

1. **Inisialisasi Queue**

- Membuat objek Queue<String> q_1004 = new LinkedList<>();.
- Queue masih kosong.

2. **Menambahkan Elemen ke Queue**

- q_1004.add("Praktikum");
- q_1004.add("Struktur");
- q_1004.add("Data");
- q_1004.add("Dan");
- q_1004.add("Algoritma");

- Queue sekarang berisi: [Praktikum, Struktur, Data, Dan, Algoritma].

3. **Membuat Iterator**

- Iterator<String> iterator = q_1004.iterator();
- Iterator digunakan untuk menelusuri elemen queue satu per satu.

4. **Iterasi Elemen Queue**

- Menggunakan loop while (iterator.hasNext()).
- iterator.hasNext() mengecek apakah masih ada elemen berikutnya.
- iterator.next() mengambil elemen saat ini dan bergerak ke elemen berikutnya.
- Semua elemen queue dicetak ke console dengan spasi.

e). ReverseData_2511531004

1. **Inisialisasi Queue**

- Membuat objek Queue<Integer> q_1004 = new LinkedList<>();.
- Menambahkan elemen 1, 2, 3 ke dalam queue.
- Kondisi awal queue: [1, 2, 3].

2. **Menampilkan Isi Queue Sebelum Dibalik**

- Output: Sebelum reverse [1, 2, 3].

3 I**nisialisasi Stack**

- Membuat objek Stack<Integer> s_1004 = new Stack<>();.
- Stack masih kosong.

4. **Memindahkan Elemen dari Queue ke Stack**

- Selama queue tidak kosong (while (!q_1004.isEmpty())):
    - Ambil elemen depan dari queue dengan remove().

    - Masukkan elemen tersebut ke stack dengan push().
- Setelah proses ini:
    - Queue kosong [].
    - Stack berisi [3, 2, 1] (karena sifat LIFO).

5. **Memindahkan Elemen dari Stack ke Queue**

- Selama stack tidak kosong (while (!s_1004.isEmpty())):
    - Ambil elemen dari stack dengan pop().
    - Masukkan elemen tersebut ke queue dengan add().
- Setelah proses ini:
    - Queue berisi [3, 2, 1].
        - **Menampilkan Isi Queue Setelah Dibalik**
- Output: data sesudah di reverse [3, 2, 1].

BAB III

PENUTUP

3.1 Kesimpulan

1. **Queue berbasis Array (**queueArray**)**

- Menggunakan indeks front dan rear dengan konsep circular queue.
- Membutuhkan pengelolaan manual terhadap kapasitas dan posisi indeks.
- Menunjukkan prinsip dasar **FIFO (First In, First Out)**.

2. **Queue berbasis LinkedList (**queueLinkedList**)**

- Lebih sederhana karena memanfaatkan Java Collection Framework.
- Operasi enqueue, dequeue, peek, dan size sudah tersedia sebagai method bawaan.
- Lebih fleksibel dibanding array karena tidak perlu menentukan kapasitas di awal.

3. **Iterasi Queue (**iterasiQueue**)**

- Menunjukkan cara menelusuri isi queue menggunakan **Iterator**.
- Memudahkan traversal tanpa harus tahu detail indeks atau struktur internal.
- Output lebih rapi dan sesuai urutan antrian.

4. **Reverse Queue dengan Stack (**reverseData**)**

- Memanfaatkan sifat **LIFO** pada stack untuk membalik urutan data queue.
- Membuktikan bahwa kombinasi struktur data dapat menghasilkan operasi tambahan yang berguna.
- Elemen pertama yang masuk menjadi elemen terakhir setelah proses reverse.

3.2 Saran

1. **Perbaikan Teknis**

- Pada queueArray, perbaiki method display agar bisa menampilkan semua elemen dengan benar, termasuk saat queue melingkar.
- Tambahkan pesan error yang lebih informatif ketika queue kosong atau penuh.

2. **Pengembangan Fitur**

- Buat menu interaktif dengan Scanner agar pengguna bisa memilih operasi enqueue, dequeue, peek, dan display secara langsung.
- Tambahkan fitur pencarian elemen dalam queue.
- Simpan hasil operasi ke file .txt atau .csv agar data tidak hilang setelah program ditutup.

3. **Pemformatan Output**

- Gunakan format tampilan yang lebih rapi, misalnya tabel atau garis pemisah.
- Konsistensi pesan output agar mudah dipahami saat praktikum.

DAFTAR PUSTAKA

1. Oracle. (2024). *Java™ Platform, Standard Edition Documentation*. Oracle. https://docs.oracle.com/javase/8/docs/api/ (docs.oracle.com in Bing)
2. Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.
3. Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (11th ed.). Pearson.
4. Wirth, N. (2006). *Algorithms and Data Structures*. Springer. (Referensi umum untuk konsep queue, stack, dan linked list).
5. GeeksforGeeks. (2025). *Queue in Java – Implementation and Examples*. https://www.geeksforgeeks.org/queue-in-java/ (geeksforgeeks.org in Bing)
6. TutorialsPoint. (2025). *Java – Collections Framework*. https://www.tutorialspoint.com/java/java_collections.htm (tutorialspoint.com in Bing)
7. Baeldung. (2025). *Guide to the Java Queue Interface*. https://www.baeldung.com/java-queue

a. Kode Program Queue_2511531004

```
package Struktur_data;

import java.util.Scanner;


public class Queue_2511531004 {


	Scanner input = new Scanner(System.in);

	int front_1004, rear_1004, size_1004;

	int capacity_1004;

	String antrian_1004 [];


	public Queue_2511531004( int capacity_1004) {

		this.capacity_1004 = capacity_1004;

		front_1004 = this.size_1004 = 0;

		rear_1004 = capacity_1004 - 1;

		antrian_1004 = new String [this.capacity_1004];



	}

	void menuAntri_1004() {

		System.out.println("======Menu Antrian Loket Pelayanan=====");

		System.out.println("1. Tambah Pelanggan");

		System.out.println("2. Hapus pelanggan");
```

```
        System.out.println("3. Reverse antrian pelanggan");
        System.out.println("4. Tampilkan Antrian");
        System.out.println("5. Keluar");

}

boolean isFull_2511531004 (Queue_2511531004 queue ) {
        return (queue.size_1004 == queue.capacity_1004);

}

boolean isEmpty_2511531004 ( Queue_2511531004 queue ) {
        return ( queue.size_1004 == 0);

}

void enqueue_2511531004 (Scanner input ) {
        if (isFull_2511531004 (this) )

        return;

        this.rear_1004 = (this.rear_1004 + 1) % capacity_1004;
        System.out.println("Masukkan Nama Pelanggan : ");
```

```
            String Pelanggan_1004 = input.nextLine();
            this.antrian_1004 [this.rear_1004] = Pelanggan_1004;
            this.size_1004 = this.size_1004 + 1;
            System.out.println("data sudah dimasukkan ke dalam Queue");
        }

        String dequeue_2511531004 () {
            if(isEmpty_2511531004 (this)) {
                System.out.println("Antrian Kosong");
                return null;
            }
            String Pelanggan_1004 = this.antrian_1004 [this.front_1004];
            this.front_1004 = (this.front_1004 + 1 ) % this.capacity_1004;
            this.size_1004 = this.size_1004 - 1;
            System.out.println(Pelanggan_1004 + "Sudah diberikan
pelayanan");
            return Pelanggan_1004;
        }

        public void reverseAntrian_1004 () {
            System.out.println("=====Hasil Reverse Antrian=====");
            for ( int i = 0; i < this.size_1004; i ++) {
                if ( i== (this.size_1004 / 2 ))
                    break;
```

```
                String temp = this.antrian_1004 [this.front_1004 + i];
                this.antrian_1004 [this.front_1004 + i ] = this.antrian_1004
[this.rear_1004];
                this.antrian_1004 [this.rear_1004 - i] = temp;

            }
            System.out.print("[");
            for ( int i = 0; i < this.size_1004; i++) {
                System.out.println(" " + this.antrian_1004[this.front_1004 +
i] + " " ) ;

                }
            System.out.print("]");
            System.out.println();
        }

        public void tampilkan_1004 () {
            System.out.println("=====Tampilkan Antrian Loket=====");
            System.out.print("[");
            for ( int i = 0; i < this.size_1004; i++) {
                System.out.println(this.antrian_1004[this.front_1004 + i ]);
            }
            System.out.print("]");
            System.out.println();
```

```
        }
}
```

b. Kode Program AntrianLoket_2511531004

```
package Struktur_data;
import java.util.Scanner;

public class AntrianLoket_2511531004 {
        public static void main (String[] args) {
                Scanner sc = new Scanner(System.in);
                Queue_2511531004 queue_1004 = new Queue_2511531004 (5);
                int pilihan_1004;


                do {
                        queue_1004.menuAntri_1004();
                        System.out.println("Pilih menu : ");
                         pilihan_1004 = sc.nextInt();
                        System.out.println();
                        sc.nextLine();


                        switch (pilihan_1004) {
```

```
            case 1 :
                queue_1004.enqueue_2511531004(sc);
                break;

            case 2 :
                queue_1004.dequeue_2511531004();
                break;

            case 3 :
                queue_1004.reverseAntrian_1004();
                break;

            case 4 :
                queue_1004.tampilkan_1004();
                break;

            case 5 :

                break;
            }

        } while (pilihan_1004 != 5);
            sc.close();
```

}

}